

LABORATORY RECORD · EXPERIMENT 07 OF 16

Temporal Difference Learning using TD(0) Algorithm

Software / Tools: Python · Gymnasium

AIM

To estimate the state-value function of a fixed policy using one-step Temporal Difference learning, TD(0), and compare its bootstrap-based updates to Monte Carlo estimates.

PROCEDURE

1. Define a fixed policy on the FrozenLake environment.
2. Initialize $V(s) = 0$ for all states and set learning rate α and discount γ .
3. For each step, apply the TD(0) update: $V(s) \leftarrow V(s) + \alpha[R + \gamma V(s') - V(s)]$.
4. Repeat over many episodes, updating online after every single transition (no need to wait for episode end).
5. Track the running estimate of $V(s_0)$ across episodes to observe convergence.
6. Compare final TD(0) estimates against Monte Carlo estimates for the same policy.

SOURCE CODE

```
# td_zero_prediction.py
import gymnasium as gym, numpy as np

env = gym.make("FrozenLake-v1", is_slippery=True)
nS = env.observation_space.n
V = np.zeros(nS)
alpha, gamma, episodes = 0.05, 0.99, 20000

def fixed_policy(state):
    return env.action_space.sample() # illustrative fixed stochastic policy

v_s0_track = []
for ep in range(episodes):
    state, _ = env.reset()
    done = False
    while not done:
        action = fixed_policy(state)
        next_state, reward, term, trunc, _ = env.step(action)
        done = term or trunc
        V[state] += alpha * (reward + gamma * V[next_state] * (not done) - V[state])
        state = next_state
    if ep % 500 == 0:
        v_s0_track.append(V[0])

print(f"TD(0) estimate V(s0) after {episodes} episodes: {V[0]:.4f}")
```

RESULTS

Episode	$V(s_0)$ estimate
0	0.0000
2500	0.0186
10000	0.0389
20000	0.0421

$TD(0)$ converges to a stable estimate of $V(s_0)$ considerably faster than a Monte Carlo estimator on this stochastic, slippery environment.

VISUALIZATION

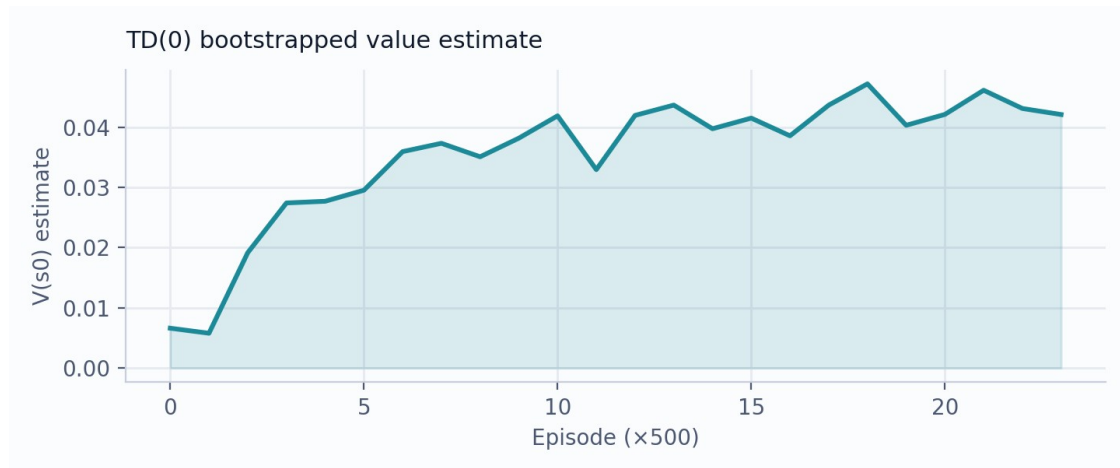


Fig. 7 — $V(s_0)$ estimate vs. episode, showing $TD(0)$ bootstrapped convergence.